GET STARTED
Welcome to SP-API Documentation
What is the Selling Partner API?
Selling Partner API Onboarding Overview
Terminology
Frequently Asked Questions

CHANGELOG
SP-API Release Notes
SP-API Deprecation Schedule
SP-API Product Metadata Updates

DIRECTORIES
SP-API Models
SP-API Seller Use Cases

# Selling Partner Appstore Authorization Workflow

Authorize a public SP-API application by using the Selling Partner Appstore flow.

A selling partner can start the authorization process for your public application from two places:

- Your application's detail page in the Selling Partner Appstore (described in this topic).
- Your website (described in Website Authorization Workflow).

In the Selling Partner Appstore workflow, selling partners use the Selling Partner Appstore to find and authorize applications that help them manage their Amazon selling business. Before a selling partner can use your application, they must authorize it to access their account information through an OAuth workflow.

This topic includes:

- A step-by-step table showing the authorization workflow.
- Detailed information about the URIs, parameters, and tokens used in the workflow.

## Selling Partner Appstore authorization workflow

The following overview diagram shows the authorization flow after the selling partner finds your app's detail page in the Selling Partner Appstore.

▶ View authorization workflow overview diagram

The following table describes the authorization process in more detail.

| Step | Actor | Action | Additional details |
|---|---|---|---|
| 1 | Selling partner | Visits your application's detail page in the Selling Partner Appstore. | N/A |
| 2 | Selling partner | Chooses **Authorize Now**, reviews the data access request, and grants permission to your application. | N/A |
| 3 | Amazon | Sends a request to your application's log-in URI (which you provided during application registration), including the following query parameters: `amazon_callback_uri`, `amazon_state`, and `selling_partner_id`. | Log-in URI details |
| 4 | Selling partner | Authenticates at your application's log-in URI, which enables your application to | Log-in URI details |

SP-API Vendor Use Cases
Seller Central URLs
Vendor Central URLs

REGISTRATION

SP-API Registration Overview

Developer Registration Request Status
Third-Party Provider Registration

Service Provider Registration and Role Approval Process Overview
Website Guidelines for Public Developers and Service Providers
User Permissions for Service Providers
Register your Application
Update your Application Information
Rotate your Application's LWA Credentials

View your Application Information and Credentials
Learn How Sellers Authorize Service Providers
Selling Partner API Roles

Policies and Agreements
About Facial Data

AUTHORIZATION

Authorize Applications
Authorize Public Applications
Selling Partner Appstore Authorization Workflow
Website Authorization Workflow
Authorize Private Applications
Renew Authorizations
Revoke Authorizations
Authorization Limits
Special Authorization Types

INTEGRATION

Building Robust Amazon SP-API Applications
SP-API SDKs

| Step | Actor | Action | Additional details |
|---|---|---|---|
| | | associate the authorization with the selling partner's account in your system. | |
| 5 | Your application | Validates the incoming request and generates a `state` parameter for security. | State parameter details |
| 6 | Your application | Sends a request back to Amazon's callback URI (provided in `amazon_callback_uri`) in step 3, including the required parameters (`amazon_state`, `state`, and optionally `redirect_uri`). | Amazon callback URI details |
| 7 | Amazon | Processes the authorization and displays a confirmation message to the selling partner in Seller Central | N/A |
| 8 | Amazon | Sends a request to your application's redirect URI with authorization information (`state`, `selling_partner_id`, `spapi_oauth_code`). | Redirect URI details |
| 9 | Your application | Validates the `state` parameter. | State parameter details |
| 10 | Your application | Sends a request to the LWA authorization server to exchange the authorization code for tokens, including required parameters (`grant_type`, `code`, `redirect_uri`, `client_id`, `client_secret`). | Authorization code exchange |
| 11 | LWA | Returns an access token and a refresh token to your application. | Token type details |
| 12 | Your application | Securely stores the refresh token for future use. | Token storage details |
| 13 | Amazon | Displays a success message to the selling partner in Seller Central. | N/A |
| 14 | Your application | (During runtime) Exchanges the refresh token for an access token and uses it to call the SP-API. | Runtime token exchange |

## Authorization details

The following sections provide additional information about specific elements of the authorization process.

### Website configuration details

Before you implement the authorization workflow, configure your website with the following security requirements:

- Set the following HTTP header to prevent cross-site request forgery:

```
HTTP
Referrer-Policy: no-referrer
```

- Create a dedicated redirect URI page (for example, `https://www.example.com/sp-api/auth`) that:
  - Receives authorization responses from Amazon.
  - Handles parameters securely.

- Validates state parameters.
- Follows your website's security best practices.

- If you support multiple regions, set up regional handling to ensure that:
  - Selling partners are directed to the correct regional Seller Central or Vendor Central URL.
  - Your redirect URI can process responses from all supported regions.

Important: Register your redirect URI when you register your application. Amazon will only send authorization responses to registered URIs. For details about redirect URIs, refer to Redirect URI details.

## Log-in URI details

A log-in URI is an endpoint that you specify when you register your application. This URI is where Amazon sends the selling partner to log in to your website so that you can index the authorization to the correct selling partner in your own system.

When Amazon sends the request to your log-in URI, it includes the following parameters:

| Parameter | Description |
|---|---|
| `amazon_callback_uri` | A URI to which your website redirects the selling partner so that they continue the authorization workflow. |
| `amazon_state` | A state value generated by Amazon to guard against cross-site request forgery attacks. |
| `selling_partner_id` | The identifier of the selling partner who is authorizing your application. |
| `version` | (Optional) A parameter that's set to `beta` if the application to be authorized is in `Draft` state. Not included in production workflows initiated from the Selling Partner Appstore. |

Example of a log-in URI:

HTTP

```
https://example.com/index.html?
amazon_callback_uri=https://amazon.com/apps/authorize/confirm/amzn1.sellerapps.
9f5a-4d13-b16c-
474EXAMPLE57&amazon_state=amazonstateexample&selling_partner_id=A3FHEXAMPLEYWS
```

## Amazon callback URI details

The Amazon callback URI is where your website redirects the selling partner so that they continue the authorization workflow.

Your website receives the callback URI as the `amazon_callback_uri` query parameter when Amazon calls your log-in URI.

When your website redirects the selling partner to the callback URI, your website includes the following parameters:

| Parameter | Description |
|---|---|
| `redirect_uri` | (Optional) A URI that receives Amazon's authorization response. Must match a URI you specified when you registered your application. If omitted, Amazon uses the first registered redirect URI. |
| `amazon_state` | The `amazon_state` value that Amazon provided in the previous request. |
| `state` | A security token that your application generates to prevent cross-site request forgery attacks. This token should be short-lived and unique to |

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK
Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js
Tutorial: Automate your SP-API Calls Using a Python SDK
Tutorial: Automate your SP-API Calls using a prebuilt C# SDK
Tutorial: Automate your SP-API Calls using prebuilt Java SDK
Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK
Tutorial: Automate your SP-API Calls using prebuilt PHP SDK
Tutorial: Automate your SP-API calls using a prebuilt Python SDK
Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application
Tutorial: Retrieve Merchant Shipping Templates
Tutorial: Grant the SP-API Permission to an Amazon SQS Queue
Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors
URL Encoding
Resolve Common HTTP and Authorization Error Codes
External Fulfillment Errors
Listings Items API Issues Troubleshooting

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?
List Your App on the Selling Partner Appstore
Edit Your Appstore Listing
Check Listing Status
Appstore Ratings and Reviews
Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview
Amazon Selling Partner API Guard Implementation Guide

| Parameter | Description |
|---|---|
| | each authorization request. |
| `version` | (Optional) A parameter that you must set to `beta` if the application to be authorized is in `Draft` state. If you don't include this parameter, the workflow authorizes an application that is published on the Selling Partner Appstore. |

For security, your application should:

- Generate a unique `state` token for each request to prevent cross-site request forgery (CSRF) attacks.
- Set the `Referrer-Policy: no-referrer` HTTP header.
- Validate the state parameter in the response.

Example of a production callback URI:

```
HTTP
https://amazon.com/apps/authorize/confirm/amzn1.sellerapps.app.2eca283f-9f5a-4d13-b16c-474EXAMPLE57?redirect_uri=https://example.com/landing.html&amazon_state=amazonstateexample&s
```

Example of a callback URI for authorizing an application that's in `Draft` state:

```
HTTP
https://amazon.com/apps/authorize/confirm/amzn1.sellerapps.app.2eca283f-9f5a-4d13-b16c-474EXAMPLE57?redirect_uri=https://example.com/landing.html&amazon_state=amazonstateexample&s
```

## Redirect URI details

A redirect URI is an endpoint that receives Amazon's authorization response after the selling partner grants permission.

When Amazon sends the request to your redirect URI, it includes the following parameters:

| Parameter | Description |
|---|---|
| `state` | The state value that your application generated and included in the callback request. |
| `selling_partner_id` | The identifier of the selling partner who authorized your application. |
| `spapi_oauth_code` | The Login with Amazon (LWA) authorization code. Your application exchanges this for an LWA refresh token. |

Example of a redirect URI:

```
HTTP
https://client-example.com?state=state-example&selling_partner_id=sellingpartneridexample&spapi_oauth_code=spapioauthc
```

When your application receives this request, it should:

- Validate the `state` parameter to ensure that it matches the one you generated.
- Securely store the `selling_partner_id` if your application needs to track or manage selling partner-specific data.
- Promptly exchange the `spapi_oauth_code` for an LWA refresh token (within five minutes).

VAT Calculation Service

Amazon Seller Data Access

Best Practices

A+ CONTENT API

A+ Content API

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API

Amazon Warehousing and Distribution API v2024-05-09 Reference

APP INTEGRATIONS API

App Integrations API

Important security considerations:

- The LWA authorization code ( `spapi_oauth_code` ) expires after five minutes. Exchange it for a refresh token before it expires.
- If the entire authorization process takes longer than 10 minutes, the workflow may break. In this case, the selling partner should restart the authorization process from the Selling Partner Appstore.

For more information on exchanging the authorization code for a refresh token, refer to Authorization code exchange details.

## Authorization URI details

The authorization URI goes to a consent page within Seller Central or Vendor Central. Although selling partners normally authorize your application through the Selling Partner Appstore, you use the authorization URI to test your authorization workflow before you publish your application.

If you test with a selling partner, make sure to use the marketplace URL that matches their Seller Central or Vendor Central region.

**To construct your Seller Central authorization URI for testing:**

1. Start with the Seller Central URL for your target marketplace.
2. Find your application ID in Seller Central or the Solution Provider Portal.
3. Combine the Seller Central URL with `/apps/authorize/consent?application_id={your application ID}` .
4. If you want to test an application that is in the `Draft` state, add `version=beta` . (You can also test without `version=beta` ; in that case, you are authorizing the published version of the application.)

   Example of a Seller Central authorization URI for testing a `Draft` application in the US marketplace:

   ```
   HTTP

   https://sellercentral.amazon.com/apps/authorize/consent?
   application_id=amzn1.sellerapps.app.0bf296b5-36a6-4942-a13e-
   EXAMPLEfcd28&version=beta
   ```

   Example of a Seller Central authorization URI for testing a `Draft` application in the Mexico marketplace:

   ```
   HTTP

   https://sellercentral.amazon.com.mx/apps/authorize/consent?
   application_id=amzn1.sellerapps.app.0bf296b5-36a6-4942-a13e-
   EXAMPLEfcd28&version=beta
   ```

**To construct your Vendor Central authorization URI for testing:**

1. Start with the Vendor Central URL for your target marketplace.
2. Find your application ID in Vendor Central or the Solution Provider Portal.
3. Combine the Vendor Central URL with `/apps/authorize/consent?application_id={your application ID}` .
4. If you want to test an application that is in the `Draft` state, add `version=beta` . (You can also test without `version=beta` ; in that case, you are authorizing the published version of the application.)

APPLICATION MANAGEMENT API
Application Management API
CATALOG ITEMS API
Catalog Items API
Catalog Items API Rate Limits
Catalog Items API v2022-04-01 Reference
CUSTOMER FEEDBACK API
Customer Feedback API
Customer Feedback API Rate Limits
DATA KIOSK
Data Kiosk API
Data Kiosk Schema Explorer User Guide
Data Kiosk Query Processing Finished Notification
Building Data Kiosk workflows guide
Vendor Analytics Dataset Use Case Guide
Schedule Data Kiosk Queries
DELIVERY BY AMAZON API
Delivery by Amazon API
EASY SHIP API
Easy Ship API
EXTERNAL FULFILLMENT
External Fulfillment Inventory API

Example of a Vendor Central authorization URI for testing a `Draft` application in the US marketplace:

```
HTTP
https://vendorcentral.amazon.com/apps/authorize/consent?
application_id=amzn1.sellerapps.app.0bf296b5-36a6-4942-a13e-
EXAMPLEfcd28&version=beta
```

Example of a Vendor Central authorization URI for testing a `Draft` application in the Mexico marketplace:

```
HTTP
https://vendorcentral.amazon.com.mx/apps/authorize/consent?
application_id=amzn1.sellerapps.app.0bf296b5-36a6-4942-a13e-
EXAMPLEfcd28&version=beta
```

## State parameter details

Two different state parameters are used in the authorization flow to prevent cross-site request forgery attacks:

| Parameter | Description | Who Creates It | Who Validates It |
|---|---|---|---|
| `amazon_state` | A security token that Amazon generates and sends to your application as a query parameter of the log-in URI. | Amazon | Your application must include it unchanged in your callback request. |
| `state` | A security token that your application generates and includes as a query parameter to the callback URI. | Your application | Your application validates it when Amazon returns it as a query parameter of the redirect URI. |

Your application should:

- Generate a unique state token for each authorization request.
- Store the state token securely until Amazon's response.
- Verify that the state in Amazon's response matches your stored value.
- Reject the authorization if the state doesn't match.

For more information about preventing cross-site request forgery attacks and generating secure state tokens, refer to Cross-site Request Forgery.

## Token type details

A token is a secure digital key that represents authorization to access SP-API resources on behalf of a selling partner. There are two types of tokens:

- **Access token:** A short-lived credential that your application uses to make SP-API calls on behalf of a selling partner.
- **Refresh token:** A long-lived credential that your application uses to get new access tokens without requiring the selling partner to re-authorize.

## Token storage details

When you store refresh tokens, follow these security best practices:

- Store tokens in a secure, encrypted database.
- Never store tokens in client-side code or logs.

External Fulfillment Returns API
External Fulfillment Shipping API
External Fulfillment Errors
External Fulfillment APIs Rate Limits
FULFILLMENT BY AMAZON (FBA)
FBA Inventory API
FBA Inventory Dynamic Sandbox Guide
FEEDS API
Feeds API
Feed Type Values
Feeds JSON Schemas
Feeds API Best Practices
Feeds API FAQ
Feeds API Rate Limits
FINANCES
Finances API
Transfers API
FULFILLMENT INBOUND API
Fulfillment Inbound API

- Never share or expose tokens in URLs.
- Implement secure backup and recovery procedures for your token database.
- Create an access control system that restricts token access to only the necessary parts of your application.
- Implement a process to handle token rotation and revocation.
- Track token age and prompt selling partners to reauthorize annually.

## Authorization code exchange details

After receiving an authorization code ( `spapi_oauth_code` ) from the redirect URI, your application exchanges the authorization code for tokens by sending a `POST` request to the Login with Amazon (LWA) authorization server ( `https://api.amazon.com/auth/o2/token` ) with the following parameters:

| Parameter | Description |
|---|---|
| `grant_type` | The type of access grant requested. Must be `authorization_code` . |
| `code` | The authorization code ( `spapi_oauth_code` ) that Amazon sent to your redirect URI. |
| `redirect_uri` | The redirect URI where you received the authorization code. Must match the URI you specified when registering your application. |
| `client_id` | Your application's client ID, which you can view in Seller Central, Vendor Central, or the Solution Provider Portal. For details, refer to View your Application Information and Credentials. |
| `client_secret` | Your application's client secret, which you can view in Seller Central, Vendor Central, or the Solution Provider Portal. For details, refer to View your Application Information and Credentials. |

**Example request to exchange an authorization code for tokens:**

HTTP

```
POST /auth/o2/token HTTP/1.1
Host: api.amazon.com
Content-Type: application/x-www-form-urlencoded;charset=UTF-8

grant_type=authorization_code&code=SplxlOexamplebYS6WxSbIA&redirect_uri=https:/
```

The LWA server returns a JSON response with the following parameters:

| Parameter | Description |
|---|---|
| `access_token` | An access token that you can use immediately for SP-API calls. For more information, refer to Connect to the Selling Partner API. |
| `token_type` | Token type. Set to `bearer` . |
| `expires_in` | Number of seconds until the access token expires (typically 3600). |
| `refresh_token` | A long-lived token that you can exchange for new access tokens. Store this securely. For more information, refer to Connect to the Selling Partner API. |

**Example response that contains the tokens:**

JSON

```
{
  "access_token": "Atza|IQEBLjAsAexampleHpi0U-**************",
  "token_type": "bearer",
  "expires_in": 3600,
```

Fulfillment Inbound API FAQ
Migrating Fulfillment Inbound workflows
Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide
Fulfillment Outbound API

Fulfillment Outbound API Rate Limits
MCF Best Practices

INVOICING

Invoices API

Invoices API FAQ

LISTINGS

Listings Items API

```
  "refresh_token": "Atzr|IQEBLzAtAhexamplewVz2Nn6f2y-tpJX2DeX"
}
```

Store the refresh token securely. For security best practices, refer to Token storage details.

### Runtime token exchange details

During runtime, your application exchanges a refresh token for an access token, except in the case of grantless operations. Note that restricted data operations indirectly require an access token. (To get a restricted data token (RDT), you call the Tokens API with the access token; then you can call the restricted operation.)

To exchange a refresh token for an access token, your application sends a `POST` request.

| Parameter | Description |
|---|---|
| `grant_type` | The type of access grant requested. Must be `refresh_token`. |
| `refresh_token` | The refresh token you received and stored during the authorization process. |
| `client_id` | Your application's client ID, which you can view in Seller Central, Vendor Central, or the Solution Provider Portal. For details, refer to View your Application Information and Credentials. |
| `client_secret` | Your application's client secret, which you can view in Seller Central, Vendor Central, or the Solution Provider Portal. For details, refer to View your Application Information and Credentials. |

**Example request to exchange a refresh token for an access token:**

HTTP

```
POST /auth/o2/token HTTP/1.1
Host: api.amazon.com
Content-Type: application/x-www-form-urlencoded;charset=UTF-8

grant_type=refresh_token&refresh_token=Atzr|IQEBLzAtAhexample&client_id=foodev&
```

The LWA server returns a JSON response with the following parameters:

| Parameter | Description |
|---|---|
| `access_token` | The access token to use in SP-API calls. |
| `token_type` | Token type. Set to `bearer`. |
| `expires_in` | Number of seconds until this access token expires (typically 3600). |

**Example response that contains an access token:**

JSON

```
{
  "access_token": "Atza|IQEBLjAsAexampleHpi0U-Dme37rR6CuUpSR",
  "token_type": "bearer",
  "expires_in": 3600
}
```

Use the returned access token in the authorization header of your SP-API requests. For more information about making SP-API calls, refer to Connect to the Selling Partner API.

You can also use an SDK to help you with this step. For details, refer to SP-API SDK.

## Test your authorization workflow

Before you publish your application, test the authorization workflow to ensure that your application correctly exchanges parameters with Amazon.

To test your authorization workflow, take the following steps:

1. Verify that your application is in `Draft` state.
2. Construct the authorization URI. For details, refer to Authorization URI details. Because you're testing with a `Draft` application, include the `version=beta` parameter in the authorization URI. (If you test without `version=beta`, you are authorizing the published version of your application.)
3. Instead of having selling partners find your application in the Selling Partner Appstore, you can test by navigating directly to the authorization URI.

   If you test with a selling partner, make sure to use the marketplace URL that matches their Seller Central or Vendor Central region.
4. After successful testing, prepare for production:
   1. List your application in the Selling Partner Appstore, which changes its state from `Draft` to `Published`.
   2. Remove the `version=beta` parameter from your callback URI handling. This allows any selling partner to authorize your published application through the Selling Partner Appstore.

Updated 19 days ago

← Authorize Public Applications

Website Authorization Workflow →

Listings Items API Rate Limits
Listings Restrictions API
Listings Restrictions API Rate Limits
Manage Product Listings with the Selling Partner API
Building Listings Management Workflows Guide
Understanding Amazon listing status and seller-fulfilled inventory management
Listings Management Workflow Migration
Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases
Listings APIs FAQ
Product Type Definitions API
Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API

MESSAGING API

Messaging API

NOTIFICATIONS API

Notifications API

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification

ORDERS API

Orders API

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API

PRODUCT PRICING API

Product Pricing API

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide

Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >